

Lab02 - ME315

Fernanda Caputo Borges, RA: 277153

```
library(readr)
library(dplyr)
library(tidyr)
library(stringr)
```

1

Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)?

As estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada são o total de ocorrências de atrasos e o número n referente ao total de vôos.

2

Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
- A função deve receber apenas dois argumentos:

- f) **input**: o conjunto de dados (referente ao lote em questão);
- ii) **pos**: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  input %>% filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
    filter(!is.na(DAY), !is.na(MONTH), !is.na(AIRLINE), !is.na(ARRIVAL_DELAY)) %>%  
    group_by(DAY, MONTH, AIRLINE) %>%  
    summarise(atrasos = sum(ARRIVAL_DELAY>10), total = n())}
```

3

Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- Configure o tamanho do lote (chunk) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
- Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
my_cols <- cols_only(DAY = "i", MONTH = "i", YEAR = "i", AIRLINE = "c", ARRIVAL_DELAY = "d")  
  
dados <- read_csv_chunked("flights.csv.zip",  
  callback=DataFrameCallback$new(getStats),  
  chunk_size=100000, col_types=my_cols)
```

4

Crie uma função chamada `computeStats` que:

- Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- Retorne as informações em um tibble contendo apenas as seguintes colunas:
- Cia: sigla da companhia aérea;

- ii) Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
- iii) Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
library(lubridate)
```

```
computeStats <- function(dados) {  
  dados %>% group_by(DAY, MONTH, AIRLINE) %>%  
    summarise(total_atrasos = sum(atrasos), total_voos = sum(total), .groups = "drop") %>%  
    mutate(Cia = AIRLINE, Data = make_date(2015, MONTH, DAY),  
           Perc = total_atrasos/total_voos) %>% select(Cia, Data, Perc)  
}
```

5

Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a) Instale e carregue os pacotes `ggcal` e `ggplot2`.

```
library(devtools)  
library(ggplot2)  
devtools::install_github("jayjacobs/ggcal")  
library(ggcal)
```

- b) Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

```
pal <- scale_fill_gradient(low="#4575b4", high="#d73027")
```

- c) Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- d) Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
- ii) Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.

iii) Retornar para o usuário a base do calendário criada acima.

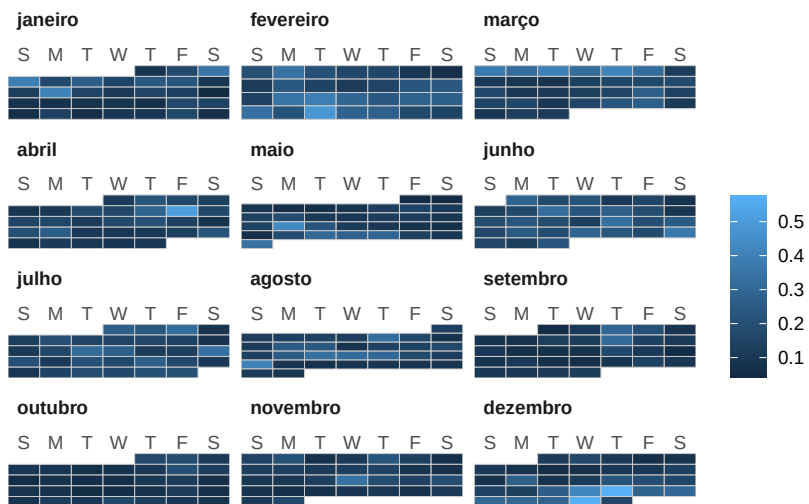
```
baseCalendario <- function(stats, cia) {
  subconj <- stats %>% filter(Cia == cia)
  baseCal <- ggcal(subconj$Data, subconj$Perc)
  return(baseCal)
}
```

d) Executar a função baseCalendario para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: cAA, cDL, cUA e cUS.

```
final_stats <- computeStats(dados)
cAA <- final_stats %>% baseCalendario("AA")
cAA
```



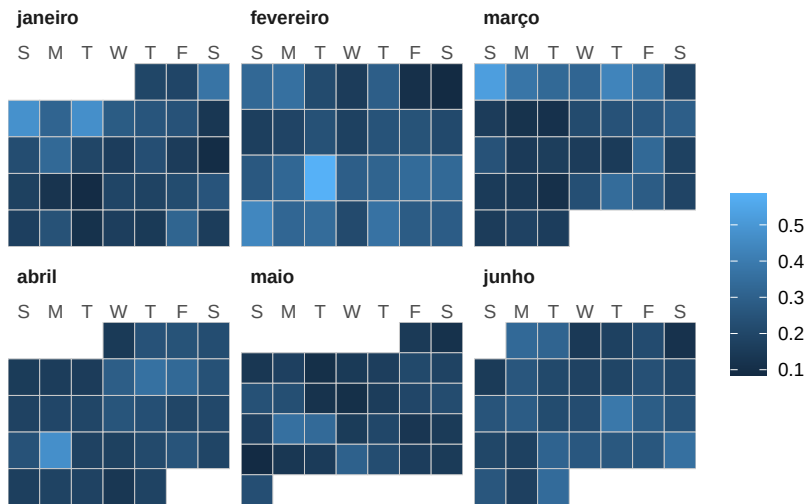
```
cDL <- final_stats %>% baseCalendario("DL")
cDL
```



```
cUA <- final_stats %>% baseCalendario("UA")
cUA
```



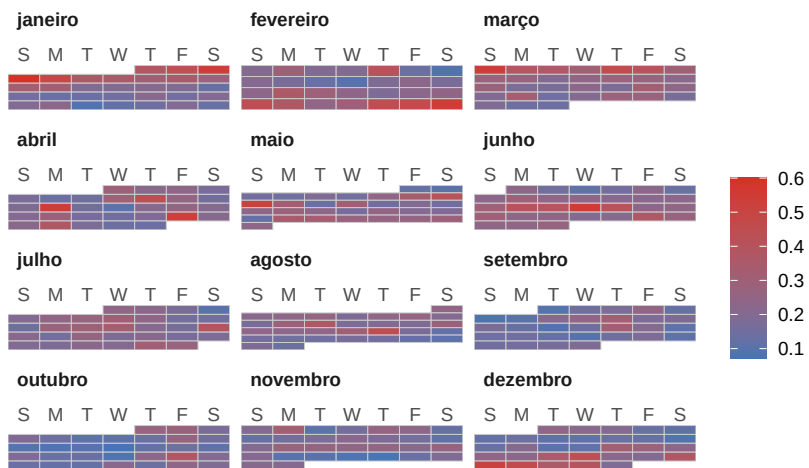
```
cUS <- final_stats %>% baseCalendario("US")
cUS
```



- e) Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do ggplot2. Lembre-se de adicionar um título utilizando o comando ggtitle. Por exemplo, cXX + pal + ggtitle("Título").

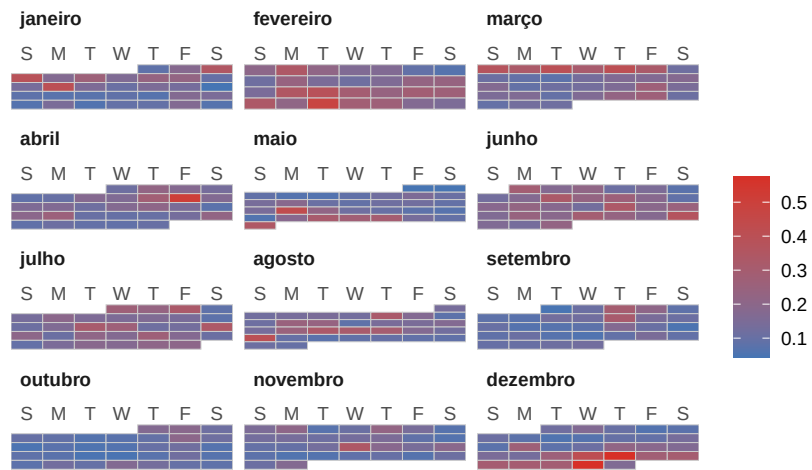
```
cAA + pal + ggtitle("Percentual de atrasos na Cia. Aérea AA")
```

Percentual de atrasos na Cia. Aérea AA



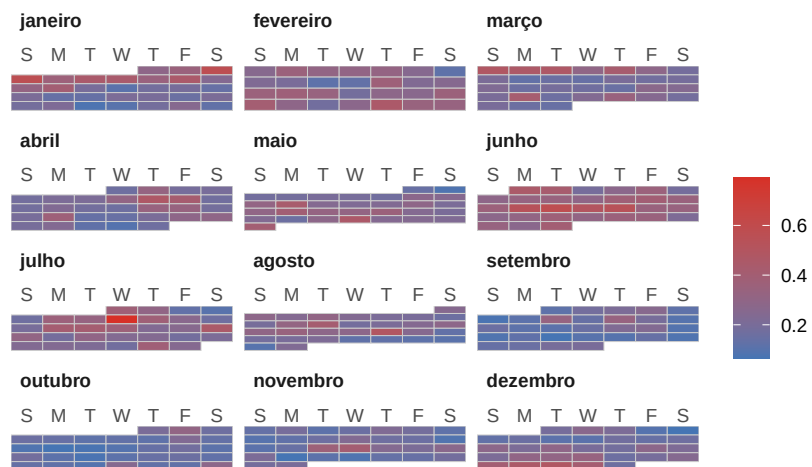
```
cDL + pal + ggtitle("Percentual de atrasos na Cia. Aérea DL")
```

Percentual de atrasos na Cia. Aérea DL



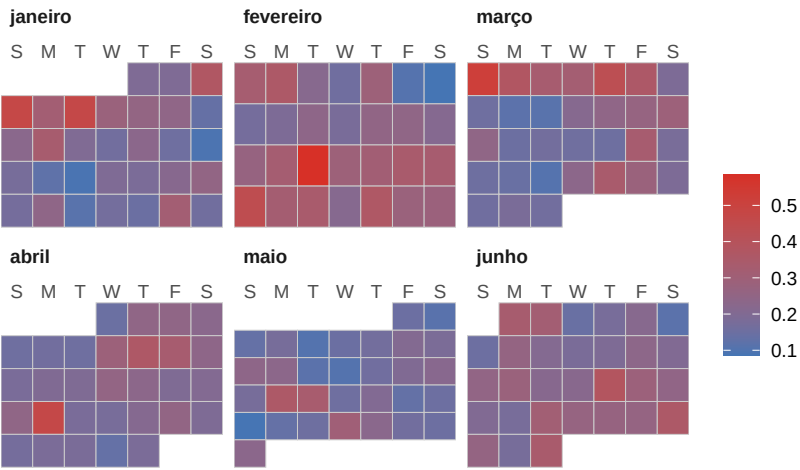
```
cUA + pal + ggtitle("Percentual de atrasos na Cia. Aérea UA")
```

Percentual de atrasos na Cia. Aérea UA



```
cUS + pal + ggtitle("Percentual de atrasos na Cia. Aérea US")
```

Percentual de atrasos na Cia. Aérea US



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

[1] "2026-08-24 23:58:47 -03"